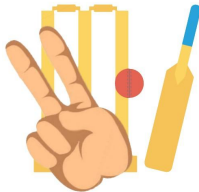


Mastering Hand Cricket

Mid-Stage Presentation

Aminesh Gogia, Pranjal Patel, Arijit Paul

March 6, 2026

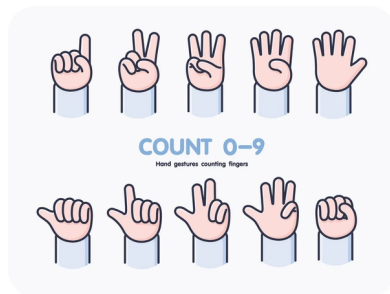


Hand Cricket: Brief Introduction

Popular game among school-going kids.

Simplest Version:

- Both players simultaneously play a number from 1 to 6.
- If same, batsman out, innings over
- If different, batsman scores, innings continue
- **Batsman in second innings:**
 - scores more → wins.
 - scores the same → draw.
 - scores less → loses.



Can be modelled as **Repeat-Until-Collision (RUC)** and **Imperfect Information Extensive Form (IIEF)** game.

- **Nash Equilibria of Two-Player Matrix Games Repeated Until Collision:**

- Establishes the existence of Nash equilibrium strategies for single-round payoffs.
- Extends results to games terminating after $w (> 1)$ collisions.

- **Infinite Over Analysis for both innings (StackExchange):**

- Solves the infinite-horizon version of the game.
- Uses a hybrid of:
 - **Linear Programming (LP)**
 - **Backward Induction**
- First solves for the second innings strategy, then for the first innings.

Problem Statement

- 1 **Validation & Scaling:** Verify existing NE solutions for the two-innings unlimited game. Assess scalability for higher N - symbol cases.
- 2 **Strategic Analysis:** Investigate the relationship between game state and "aggression" (e.g., probability distribution shifts toward higher symbols).
- 3 **Limited Overs:** Incorporate "balls remaining" into the state space.
- 4 **Real-time Adaptation:** Explore Tree-Search variants (e.g., MCTS) for games where rules (symbols/balls) are defined at runtime.
- 5 **Implementation:** Deploy a competitive agent on `handcricket.in`

The bot should be able to defeat a variety of human players with a significant win rate.

Addressing Proposal Feedback

Focus of initial implementation:

- Begin with computing the **Nash Equilibrium (NE)** for the **finite-horizon two-innings hand cricket game**.
- Model the game as a **zero-sum stochastic game** where:
 - Players choose symbols $i, j \in \{1, \dots, M\}$ simultaneously.
 - If $i = j$, a **collision** occurs and the innings ends.
 - Otherwise, the batting player scores i runs.
- Solve the game using **dynamic programming + matrix game NE**.

Key assumptions in implementation:

- Fixed number of balls T
- Symbol set $\{1, \dots, M\}$
- Perfect rationality (players compute equilibrium strategies)
- Zero-sum payoff structure
- Tie treated as value 0.5

Finite Horizon Game Formulation

State Variables

- t : balls remaining
- s : score in first innings
- k : runs required in second innings

Second Innings Value Function

$$V(t, k) = \begin{cases} 1 & k \leq 0 \\ 0 & t = 0, k > 1 \\ 0.5 & t = 0, k = 1 \end{cases}$$

At each state we solve the matrix game

$$A_{ij} = \begin{cases} 0 & i = j \\ V(t-1, k-i) & i \neq j \end{cases}$$

Goal: Compute mixed strategies (p, q) that form the Nash equilibrium.

First Innings Formulation

State:

- t : balls remaining
- s : current score

If the batsman gets out, the game transitions to the chase:

$$W(t, s) = 1 - V(T, s + 1)$$

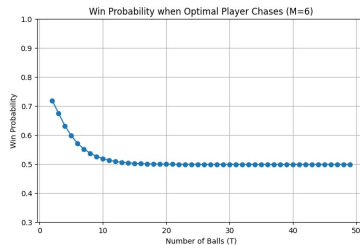
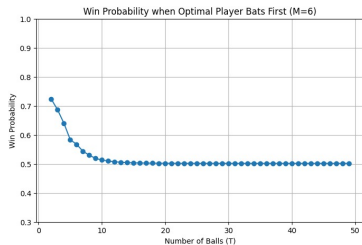
Otherwise, runs are added:

$$A_{ij} = \begin{cases} 1 - V(T, s + 1) & i = j \\ W(t - 1, s + i) & i \neq j \end{cases}$$

Solution method

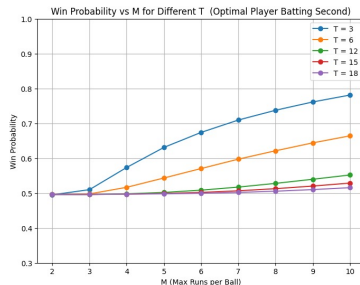
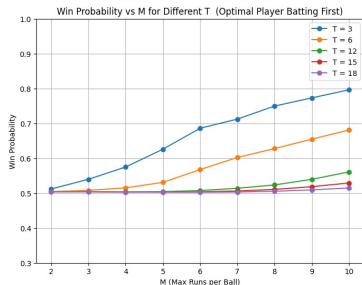
- Solve each state as a **zero-sum matrix game**
- Compute mixed strategies via **Linear Programming**
- Store equilibrium policies for each state

Analysis of the Nash agent: Theoretic Win Prob vs Random



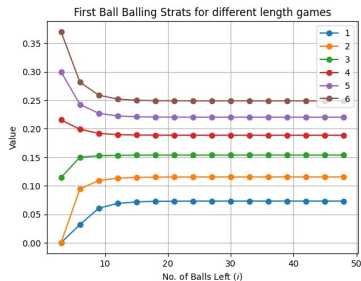
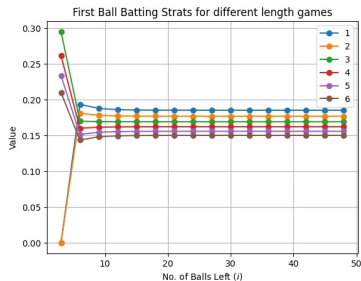
Win Prob goes to 50% on increasing the number of balls :(

Analysis of the Nash agent (contd.)



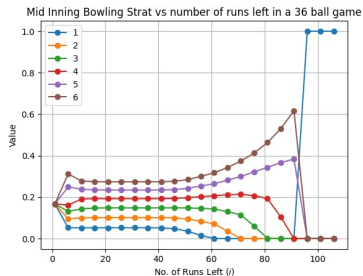
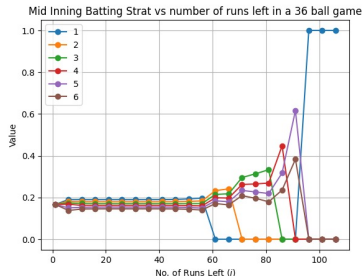
Win Rate is higher for more symbols keeping number of balls fixed!

Analysis of the Nash agent : 1st Inning Strategies



The distributions are fairly constant as the number of balls left increases :O

Analysis of the Nash agent : 2nd Inning Strategies



Again distributions are fairly constant w.r.t. no. of runs left (except at extremes!)

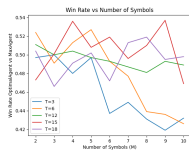
Agents used for Hand Cricket Simulation

- **Optimal** Uses Nash Equilibrium based probabilities for the different symbols during the game using backward induction to populate the game tree and play accordingly.
- **AggrPayoff** Same as Optimal but rewards finishing earlier in terms of balls left for both bowler and batter at end of match.
- **NRRFollower** Maintains the Net Run Rate (NRR) at every instance and samples uniformly from symbols above the NRR target while chasing (for both bowling and batting). Uses Optimal strategy while batting first.

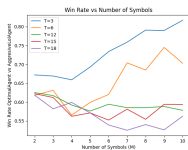
Agents used for Hand Cricket Simulation

- **PatternFollower** Plays symbols following a fixed pattern in both innings for both bowlers and batters.
- **AggresiveLolAgent** Plays symbols uniformly from the top 2/3 of the symbol range and the maximum symbol in both innings.
- **MaxAgent** Always plays the maximum symbol throughout the game.
- **RandomAgent** Plays all symbols uniformly at random throughout the game.

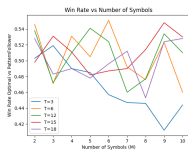
Win Rate of Optimal Agent against others while batting first (as function of M)



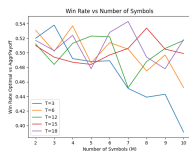
MaxAgent



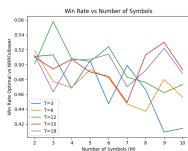
AggressiveLolAgent



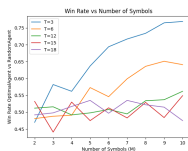
PatternFollower



AggrPayoff



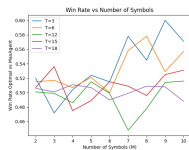
NRRFollower



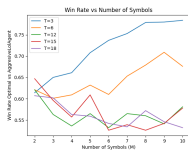
RandomAgent

where M is number of symbols

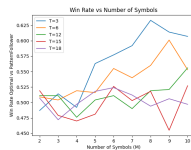
Win Rate of Optimal Agent against others while batting second (as function of M)



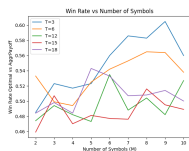
MaxAgent



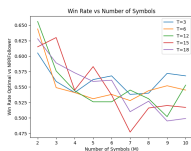
AggressiveLolAgent



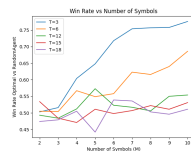
PatternFollower



AggrPayoff



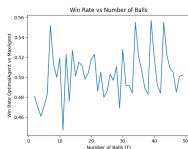
NRRFollower



RandomAgent

where M is number of symbols

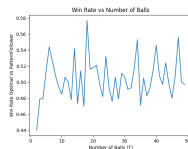
Win Rate of Optimal Agent against others while batting first (as function of T)



MaxAgent



AggressiveLolAgent



PatternFollower



AggrPayoff



NRRFollower



RandomAgent

where T is number of balls in innings

Win Rate of Optimal Agent against others while batting second (as function of T)



MaxAgent



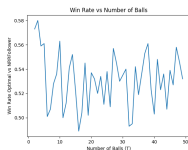
AggressiveLolAgent



PatternFollower



AggrPayoff



NRRFollower



RandomAgent

where T is number of balls in innings

Mid-Term Deliverables

Before MidSem

- ✓ Literature review
 - ✓ Nash equilibria in two-player games
 - ✓ Repeat-Until-Collision (RUC) games
 - ✓ Imperfect Information Extensive Form Games (IIEFGs)
- ✓ Deployment of a baseline agent for the online game
- ✓ Exploration of game-theoretic and RL-based approaches

By the first week of March

- ✓ Extension of existing equilibrium solutions from 6 symbols to 10 symbols

Observation

- In the finite-horizon setting ($T = 10$), the Nash equilibrium agent achieves at most $\sim 55\%$ win rate.
- Equilibrium strategies are **robust**, but not necessarily strongly dominant in short games.

Observation

- In the finite-horizon setting ($T = 10$), the Nash equilibrium agent achieves at most $\sim 55\%$ win rate.
- Equilibrium strategies are **robust**, but not necessarily strongly dominant in short games.

Implication

- Reinforcement learning could potentially improve performance.
- However, RL requires repeated interaction with the **same opponent**.

Deviation in Future Plan

Observation

- In the finite-horizon setting ($T = 10$), the Nash equilibrium agent achieves at most $\sim 55\%$ win rate.
- Equilibrium strategies are **robust**, but not necessarily strongly dominant in short games.

Implication

- Reinforcement learning could potentially improve performance.
- However, RL requires repeated interaction with the **same opponent**.

Updated Direction

- Compute the **infinite-horizon optimal strategy**.
- Compare with the **stationary equilibrium** predicted by RUC theory.
- Use the infinite-horizon setting as a **long training horizon** for RL extensions.

References



Joseph Sol Mony T.
Hand Cricket, 2023.



goriki
Playing Odd-Even Cricket: Is there a perfect strategy?



A. Murhekar and E. Sharma
Nash Equilibria of Two-Player Matrix Games Repeated Until Collision
FSTTCS, 2023.



D. Koller and N. Megiddo
The Complexity of Two-Person Zero-Sum Games in Extensive Form
Games and Economic Behavior, 1992.



K. Leyton-Brown and Y. Shoham,
Essentials of Game Theory.



K. Leyton-Brown and Y. Shoham,
Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations.

Thank You

Questions?